Clase 5 · Mapas de memoria, arranque y SRAM vs. DRAM

Fundamentos de Sistemas Embebidos · M.C. José Mauricio Matamoros de María y Campos · miércoles 9 sep 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Registros de AVR (Arduino)

Bloque	Cantidad
Registros de trabajo (la ALU solo opera sobre ellos)	32
Control de periféricos internos (puertos, timers, comparadores, ADC)	64
E/S externa, de 0x60 a 0xFF	416

- RAM integrada de 8 KB (8 bits); renunciando a registros externos (línea de *shift*) se expande hasta **64 KB**.
- Ingeniería = manipular registros, no solo funciones del IDE.

2. ESP32: memoria y búsqueda en ROM

- RISC de 32 bits con buses separados de instrucciones y datos. ROM 0 = 256 KB, ROM 1 = 128 KB; RAM 0 = 32 KB, RAM 1 = 412 KB, RAM 2 opcional; caché.
- Guarda programas y **tablas** (trigonométricas) en ROM, no en RAM como PIC o Arduino.

Búsqueda	Datos	Complejidad
Lineal	Desordenados	O(N)
Binaria	Ordenados	$O(\log_2 N)$
Hash	i = g(dato)	**O(1)**; en el ESP32 ≈ 3 multiplicaciones enteras

3. RP2040: caché e intercalado

- Las direcciones de periféricos **no pasan por caché**: sus datos cambian y se leerían desactualizados.

> **Intercalado (examen)**
>
> RAM y periféricos alternados por **retrocompatibilidad**: al agregar RAM en una revisión nueva, se pone debajo de los periféricos para no mover direcciones de programas ya compilados. **No** es por tiempos de acceso (la RAM tarda lo mismo en cualquier dirección). Sucesor: RP2350 con 512 KB de SRAM y 16 MB de Flash.

4. Mapa de ARM Cortex-M (v6M)

Región	Tamaño
Periféricos externos	1 GB
Periféricos internos	0.5 GB
RAM	0.5 GB
Flash de programa	0.5 GB

Los chips reales usan una fracción mínima; el mapa del núcleo no es el de la tarjeta.

5. Arranque de la Raspberry Pi

Física desde 0x00000000 (RAM); arriba, el bootloader con su ROM.

1. El procesador ejecuta la ROM del bootloader.
2. Inicializa el bus **SPI**.
3. Busca MicroSD o USB.
4. Copia el **Kernel** a RAM.
5. Salta a la RAM y arranca el Kernel.

Sin MicroSD: pantalla de calibración RGB ("arcoíris") y la frambuesa, como el BIOS sin disco.

6. Espacio virtual, segfault y sistemas virtuales

- Espacio de Kernel vs. de usuario; los programas ven direcciones **virtuales** paginadas.

```
int *s = NULL;
s = 1;   /* escribe en 0x00000000, donde está el Kernel */
```

→ la protección de memoria genera **segmentation fault** y mata el proceso.

- Drivers: el Kernel inspecciona el bus y carga solo los módulos compatibles (`/lib/modules`).
- **/sys, /proc, /dev** son **virtuales**: vacíos en disco; el Kernel los crea en RAM al arrancar.

7. SRAM vs. DRAM

	SRAM	DRAM
Por bit	**6 MOSFET**	**1 transistor + 1 capacitancia parásita**
Costo / densidad	Cara, baja densidad	Barata, **6×** más densa
Velocidad	Extremadamente rápida (femtosegundos)	Más lenta
Calor	Alto: escribir un 0 drena el 1 a tierra → calor	Bajo
Refresco	No	Cada **~60 ms**
Uso	Caché	RAM principal

DRAM: Robert Dennard, IBM, 1967. Error común: creer que la DRAM disipa como la SRAM.

Contexto y pendiente

- "Debug": insectos rostizados en las computadoras de bulbos. Copilot en el Kernel de Windows omitió el monitoreo de temperatura.
- Práctica 6 (ADC con Arduino); Práctica 8 (Linux propio con boot splash personalizado). Sin clase 15 y 16 sep.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
AVR	32 de trabajo, 64 de periféricos, 416 externos (0x60–0xFF)
Búsqueda	Lineal O(N), binaria O($\log_2$ N), hash O(1)
Intercalado RP2040	Retrocompatibilidad, no tiempos de acceso
Cortex-M v6M	1 GB ext., 0.5 int., 0.5 RAM, 0.5 Flash
Arranque	ROM → SPI → MicroSD/USB → Kernel a RAM → salto
NULL = 1	Segmentation fault
/sys /proc /dev	Virtuales, en RAM

SRAM vs. DRAM

SRAM: 6 MOSFET, rápida, cara, caliente, caché. DRAM: 1T + 1C, densa, barata, fría, refresco cada 60 ms, RAM principal.